

Program 1

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import fetch_california_housing

housing = fetch_california_housing(as_frame=True)
df = housing.frame

num_cols = df.select_dtypes(include=np.number).columns

# Histograms
for col in num_cols:
    df[col].hist()
    plt.title(col)
    plt.show()

# Boxplots
for col in num_cols:
    sns.boxplot(x=df[col])
    plt.title(col)
    plt.show()

# IQR Outlier Detection
for col in num_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)

    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    outliers = df[
        (df[col] < lower) |
        (df[col] > upper)
    ]

    print(col, ":", len(outliers))
```

Program 2

```
from sklearn.datasets import fetch_california_housing
import seaborn as sns
import matplotlib.pyplot as plt

df = fetch_california_housing(as_frame=True).frame

corr = df.corr()

sns.heatmap(corr)
plt.show()

sns.pairplot(df)
plt.show()
```

Program 3

```
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

X, y = load_iris(return_X_y=True)

X_pca = PCA(n_components=2).fit_transform(X)

plt.scatter(X_pca[:,0], X_pca[:,1], c=y)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("PCA on Iris Dataset")
plt.show()
```

Program 4

```
import pandas as pd

data = pd.read_csv("training_data.csv")

attr = data.iloc[:, :-1].values
target = data.iloc[:, -1].values
print(data)
print("most specific hypothesis:")

for i in range(len(target)):
    if target[i] == "yes":
        hyp = attr[i].copy()
        break

for i in range(len(target)):
    if target[i] == "yes":
        for j in range(len(hyp)):
            if hyp[j] != attr[i][j]:
                hyp[j] = '?'

print(hyp)
```

Program 5

```
import numpy as np
import pandas as pd
from sklearn.neighbors import KNeighborsClassifier

np.random.seed(42)
x = np.random.rand(100)

labels = ['Class1' if i <= 0.5 else 'Class2' for i in x[:50]] + [None]*50

df = pd.DataFrame({
    'Point': [f'x{i+1}' for i in range(100)],
    'Value': x,
    'Label': labels
})

X_train = df[df.Label.notna()][['Value']]
y_train = df[df.Label.notna()][['Label']]
X_test = df[df.Label.isna()][['Value']]

for k in [1,2,3,4,5,20,30]:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)
    df.loc[df.Label.isna(), f'Label_k{k}'] = model.predict(X_test)

print(df.tail(50))
```

Program 6

```
import numpy as np, matplotlib.pyplot as plt

np.random.seed(42)
x=np.linspace(0,2*np.pi,100)
y=np.sin(x)+0.1*np.random.randn(100)
X=np.c_[np.ones(100),x]

def lwr(z,t=0.5):
    w=np.exp(-((x-z)**2)/(2*t*t))
    W=np.diag(w)
    th=np.linalg.inv(X.T@W@X)@X.T@W@y
    return [1,z]@th

xt=np.linspace(0,2*np.pi,200)
plt.scatter(x,y,c='r')
plt.plot(xt,[lwr(i) for i in xt], 'b')
plt.title("locally weighted regrsiion")
plt.show()
```

Program 7

```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
import matplotlib.pyplot as plt

# Linear Regression
data = pd.read_csv("boston_housing.csv")

X = data[['RM']]
y = data['MEDV']

lr = LinearRegression()
lr.fit(X, y)

plt.scatter(X, y)
plt.plot(X, lr.predict(X))
plt.title("Linear Regression")
plt.show()

# Polynomial Regression (Auto MPG)
data = pd.read_csv("auto_mpg.csv")

data['horsepower'] = pd.to_numeric(
    data['horsepower'],
    errors='coerce'
)

data = data.dropna(subset=['horsepower'])

X = data[['horsepower']]
y = data['mpg']

X_poly = PolynomialFeatures(2).fit_transform(X)

pr = LinearRegression()
pr.fit(X_poly, y)

plt.scatter(X, y)
plt.plot(X, pr.predict(X_poly))
plt.title("Polynomial Regression")
plt.show()
```

Program 8

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

data = load_breast_cancer()
X, y = data.data, data.target

Xtr, Xte, Ytr, Yte = train_test_split(X, y, test_size=0.2)

m = DecisionTreeClassifier(max_depth=3).fit(Xtr, Ytr)

print("Accuracy:", accuracy_score(Yte, m.predict(Xte))*100)

pred = m.predict([X[0]])
print("Predicted Class for the new sample:", data.target_names[pred[0]])

plot_tree(m, filled=True)
plt.show()
```


Program 9

```
from sklearn.datasets import fetch_olivetti_faces
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
faces = fetch_olivetti_faces()
X = faces.data
y = faces.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2
)
model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
for i in range(15):
    plt.subplot(3,5,i+1)
    plt.imshow(X_test[i].reshape(64,64), cmap="gray")
    plt.title(f"A:{y_test[i]} P:{y_pred[i]}")
    plt.axis("off")
plt.show()
```

Program 10

```
from sklearn.datasets import load_breast_cancer
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
data = load_breast_cancer()
X = data.data
kmeans = KMeans(n_clusters=2)
kmeans.fit(X)
labels = kmeans.labels_
plt.scatter(X[:,0], X[:,1], c=labels)
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```